HOMEWORK 1 · APPLIED AI/ML

Driving ComfyUI with agents

Image generation you can script, automate and hand to an LLM

Mo · [Date]



AGENDA · 40 MIN

Four parts, two live demos

01

What & why

ComfyUI, and why it beats a prompt box

≈ 7 min

02

How it works

Nodes, CLIP, diffusion + demo 1

≈ 13 min

03

Agent control

JSON, the API, tools + demo 2

≈ 14 min

04

Wrap-up

Pitfalls, design impact, Q&A

≈ 6 min

WHAT IS COMFYUI?

A node editor for image models

Open source

Free, runs on your own GPU, with a huge library of community nodes.

Node-based

Every step of generation is a box you can see, wire and swap.

Model-agnostic

Stable Diffusion, Flux and many image and video models.

Think of it as layers in Photoshop, but for the generation pipeline itself.

WHY BOTHER?

Prompt box vs. pipeline

	Midjourney-style tools	ComfyUI
Control	One prompt + a few settings	Every step is a node you can change
Runs on	The vendor’s cloud	Your GPU, or a cloud GPU you choose
Cost	Subscription	Free software; you pay for hardware
Repeatable	Partly	Same workflow + seed → same image
Automation	Limited	Full HTTP API: workflows are JSON

PART 2 · HOW IT WORKS

The default workflow, node by node

01
Load Checkpoint
Loads the model, its CLIP and its VAE

→

02
CLIP Text Encode
Turns your prompt into numbers

→

03
KSampler
Removes noise step by step, guided by the prompt

→

04
VAE Decode
Converts the latent into pixels

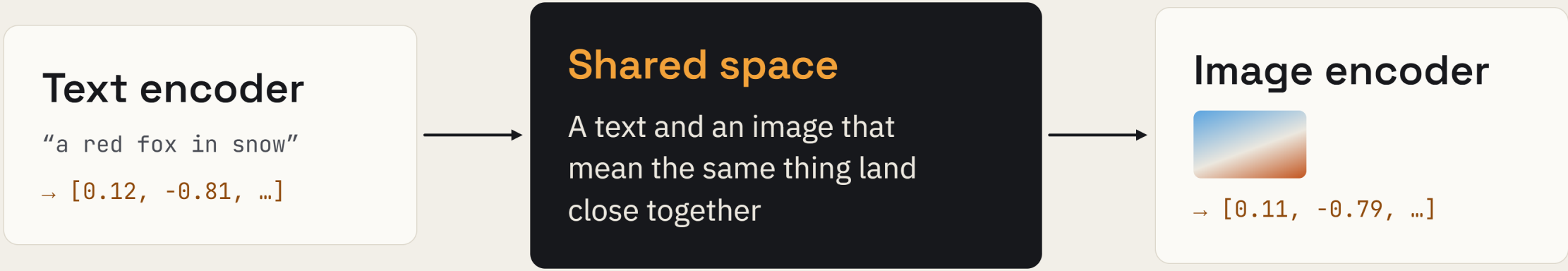
→

05
Save Image
Writes the PNG, with the workflow embedded

Plus **Empty Latent Image**: the blank, noisy canvas the sampler starts from. It also sets the width and height.

NODE 02 UP CLOSE

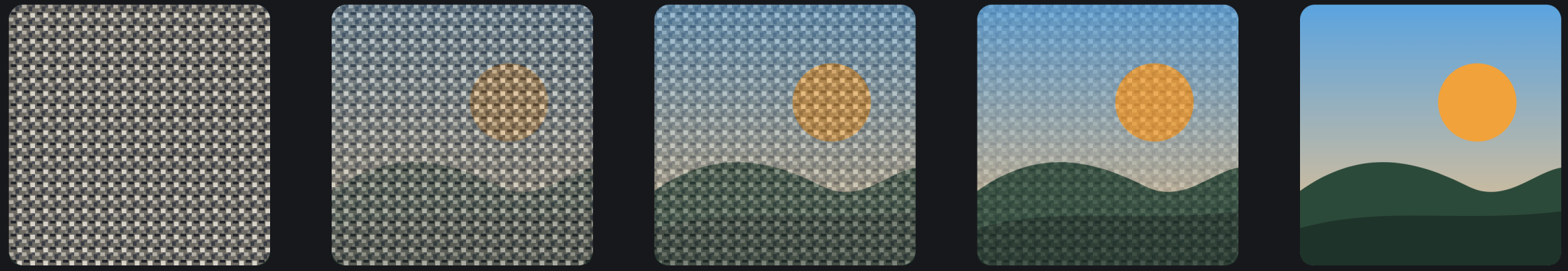
CLIP: the bridge between words and pixels



Trained on about 400 million image–caption pairs (OpenAI, 2021). In ComfyUI, the text side turns your prompt into the vector the sampler **steers by at every step**.

NODE 03 UP CLOSE

Diffusion: from noise to image



step 0 · noise | step 5 | step 10 | step 15 | step 20 · done

Each step the model predicts which part is noise and removes a little of it. It works on a small **latent** (128×128 for a 1024 px image), and the VAE scales it up to pixels at the end.

KSAMPLER SETTINGS

The five knobs that matter

Setting	What it does	Start with
Seed	Picks the starting noise	Lock it while tweaking the prompt
Steps	Number of denoising passes	20–30; more is slower, not better
CFG	How strictly to follow the prompt	5–8 for SD; too high looks burnt
Sampler	The denoising algorithm	euler or dpmpp_2m are safe
Denoise	How much of the input to repaint	1.0 new image · 0.4 restyle a photo

LIVE DEMO 1 · THE GUI

Build it by hand

1. Run the default workflow
2. Change only the seed: a whole new image
3. Lock the seed, change one word: a controlled edit

Every workflow is just JSON

```
{
  "6": {
    "class_type": "CLIPTextEncode",
    "inputs": {
      "text": "a red fox in snow",
      "clip": ["4", 1]
    }
  },
  "3": { "class_type": "KSampler", … }
}
```

Nodes are keys

Each node is one entry, keyed by its id.

Wires are references

["4", 1] means output 1 of node 4.

Export (API)

Saves any graph in this format.

THE LOCAL SERVER

ComfyUI speaks HTTP

`POST /prompt`

Send the workflow JSON; get a prompt_id back

→

`WS /ws`

Live progress: which node runs, which step

→

`GET /history/{id}`

When done: the file names of the outputs

→

`GET /view`

Download the image itself

Default address **http://127.0.0.1:8188**: the same server the GUI talks to. The browser window is just one client.

FROM CODE

Queue an image in ten lines

```
import json, uuid, requests

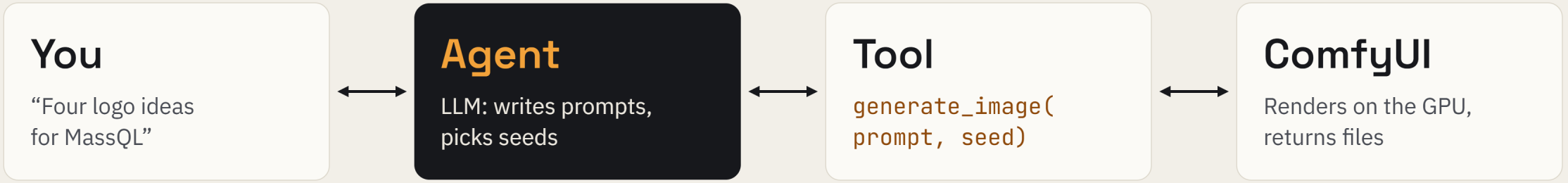
wf = json.load(open("workflow_api.json"))
wf["6"]["inputs"]["text"] = "a red fox in snow, watercolor"
wf["3"]["inputs"]["seed"] = 1234

r = requests.post("http://127.0.0.1:8188/prompt",
                  json={"prompt": wf, "client_id": str(uuid.uuid4())})
prompt_id = r.json()["prompt_id"]
# …poll /history/{prompt_id}, then download via /view
```

That’s the whole integration: **edit the JSON, POST it, collect the file.**

FROM AN AGENT

Wrap the script as a tool



The LLM is the art director

Plans variations, writes detailed prompts, judges and iterates

ComfyUI is the studio

Runs the models; never needs to understand the brief

Plug it in via

Tool use / function calling, or an MCP server

LIVE DEMO 2 · THE AGENT

“Design four logo ideas for MassQL”

One sentence in. The agent writes four prompts, calls ComfyUI four times, and explains its choices.

PART 4 · WRAP-UP

Pitfalls and practical tips

GPU memory

8 GB+ VRAM is comfortable; less works with smaller models.

Describe, don’t command

Image models match words to visuals; they don’t follow orders.

Text in images

Many models garble letters; add type later in a design tool.

Lock the seed

Change one thing at a time, or you can’t see what helped.

Read the licence

Each model has its own terms for commercial use.

Pre-render demos

Generation can take minutes; keep backup images ready.

DISCUSSION

What does this mean for designers?

01 If an agent makes 50 variations, is the job now **choosing**?

02 Who owns a style when it lives in a workflow file?

03 Which part of your work would you never hand to a pipeline?

THANK YOU

Questions?

ComfyUI

ComfyUI on GitHub · docs.comfy.org

CLIP paper

Radford et al., 2021: Learning Transferable Visual Models From Natural Language Supervision

Demo code

comfy_agent_demo.py · [link in the homework channel]